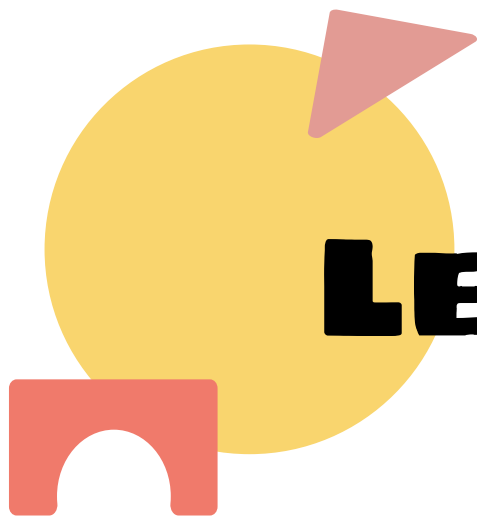




LEKTION 13

SINGLETON

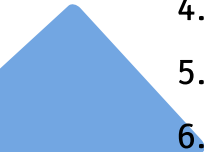
MÅL

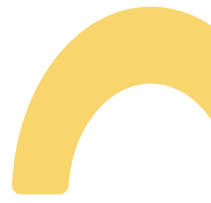
EFTER ENDT UNDERVISNING KAN DEN STUDERENDE:

- Redegøre for brugen af Singleton
- Udfærdige en singleton i C#
- Forstå de tre byggesten i singleton



AGENDA

- 
1. Beskrivelse
 2. Problem
 3. Behov
 4. Løsning
 5. Opgave
 6. Næste gang



BESKRIVELSE

Vi har en web applikation med frontend, backend og database. Brugeren skal have mulighed for at sætte forskellig indstillinger.

Vi har 5 central klasser:

- API
- Service
- Logger
- Database
- Auth

De skal alle bruge de samme settings Nogle kører lokalt, nogle i test, nogle i prod

1. Hvordan får alle adgang til de samme settings?
2. Hvad gør I, hvis settings ændrer sig?
3. Hvem “ejer” settings?
4. Hvor skal de loades?

BEHOV

LØSNING A: “ALLE LOADER SELV”

- ✓ Nemt lokalt
- ✗ Duplikeret logik
- ✗ Risiko for forskellige værdier (“dev” vs “prod”)
- ✗ Performance/spild (læser fil/miljøvariabler mange gange)



BEHOV

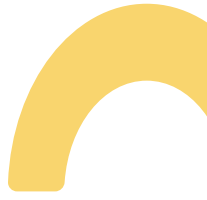
LØSNING B: “SEND CONFIG RUNDT I ALLE CONSTRUCTORS”



✓ Eksplicit

✗ “Rører” hele koden (parameter-spaghetti)

✗ Små ændringer kræver ændringer mange steder



BEHOV

LØSNING C: “GLOBAL VARIABLE / STATIC FIELDS OVERALT”

- ✓ Hurtigt
- ✗ Ingen kontrol/validering
- ✗ Svært at teste/overstyre
- ✗ Skjult afhængighed



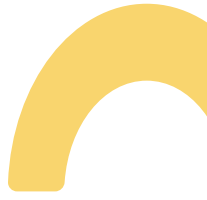
KORREKT LØSNING

VI ØNSKER:

- En fælles instans (mere kontrol)
- Kontrolleret adgang (En gang)
- En sandhed (kilde til truth)



HVORDAN LAVER V I EN KONTROLLERET INSTANS MED KONTROLLERET ADGANG?

1. private instans -> kontrolleret
 2. privat constructor -> kontrolleret
 3. static getInst -> returnere den kontrolleret instans
- 
- 

KORREKT LØSNING

KODE

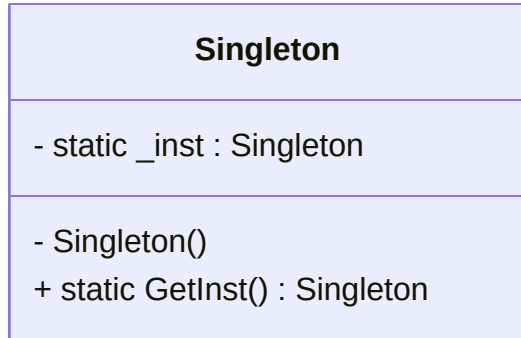
```
class Singleton
{
    private static Singleton _inst = null;

    private Singleton()
    {

    }

    public static Singleton GetInst()
    {
        if ( _inst == null)
        {
            _inst = new Singleton();
        }
        return _inst;
    }
}
```

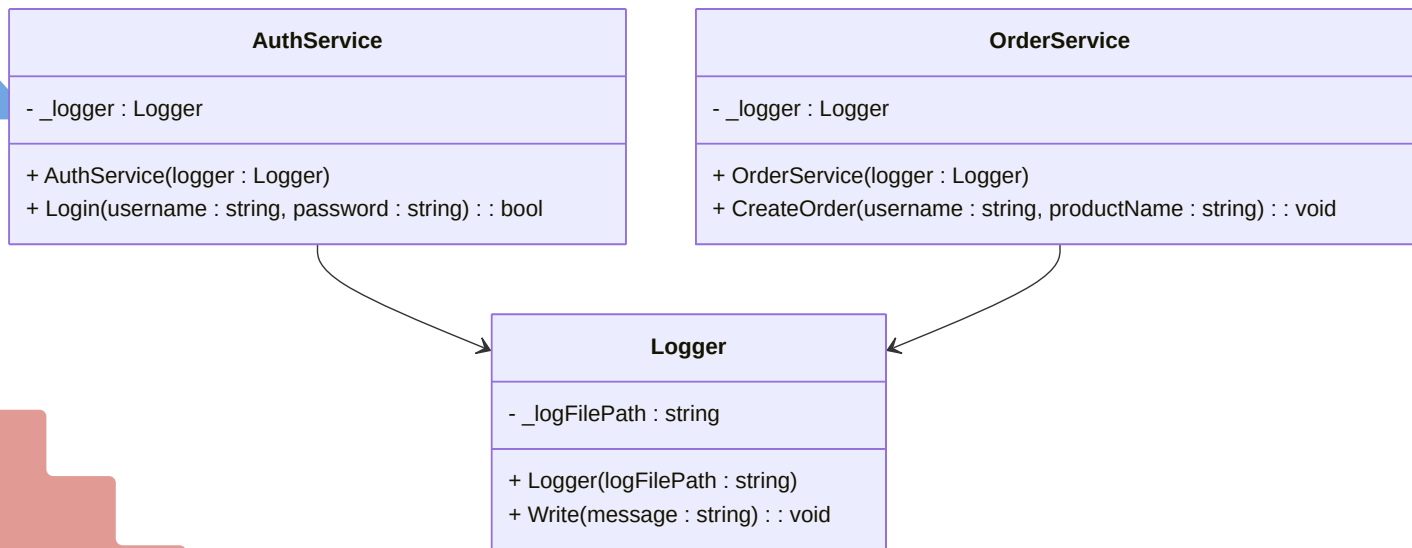
KLASSEDIAGRAM



OPGAVE 1

Udvid koden med en Singleton - findes på github under solution.

1. Færdiggør klassediagram ved at tilføje Singleton.
2. Lav rettelser i koden, så alle services bruger samme logger-instans, og der skrives til logfilen.





NÆSTE GANG

VI SKAL SNAKKE FACTORY PATTERN - DET BLIVER PÅ TORSDAG. DET ER OGSÅ TORSDAG VI SKAL VI SKAL AFLEVERE PORTFOLIO 3, OG FOR PORTFOLIO 4.

